HW1a: MNIST Classification with MLP

Omar Alaksh
Student ID: 38192

GitHub: https://github.com/OmarOksh/CS515_Deep_Learning

March 10, 2026

Contents

1 Problem Definition **2**

2 Dataset **2**

3 Neural Network Model **2**

4 Training Framework **3**

5 Numerical Results and Ablations **3**
5.1 Impact of MLP Architecture (Depth and Width) . . . 3
5.2 Impact of Activation Functions . . . 4
5.3 Impact of the Dropout Module . . . 4
5.4 Impact of Batch Normalization . . . 5
5.5 Impact of Regularization . . . 5
5.5.1 L2 Regularization (Weight Decay) . . . 5
5.5.2 L1 Regularization . . . 5
5.6 Impact of the Training Framework . . . 6
5.6.1 LR Scheduler Comparison . . . 6
5.6.2 Early Stopping Analysis . . . 6
5.7 t-SNE Feature Visualization . . . 6

6 Training and Validation Curves **7**

7 Summary **10**

1 Problem Definition

The objective of this project is to implement a Multi-Layer Perceptron (MLP) architecture to perform a 10-class classification task on the MNIST dataset. The scope of the work includes systematic ablation studies across architectural choices (depth, width), activation functions (ReLU, GELU), regularization techniques (Dropout, L1, L2), batch normalization placement, and training framework components (learning rate schedulers, early stopping). All experiments share a common codebase organized according to the prescribed structure using `argparse`, Python dataclasses, type hints, and docstrings.

2 Dataset

The MNIST dataset consists of 70,000 grayscale images of handwritten digits (0–9), each of size 28×28 pixels. The dataset is split into 60,000 training images and 10,000 test images. The classes are approximately balanced, with each digit contributing roughly 6,000–7,000 training samples. All images are normalized using the dataset-wide mean ($\mu = 0.1307$) and standard deviation ($\sigma = 0.3081$) before being fed into the network. The 2D images are flattened into 784-dimensional vectors via `nn.Flatten` for input to the MLP.

3 Neural Network Model

The baseline MLP architecture is constructed using a `nn.Sequential` container with the following block pattern repeated for each hidden layer:

`Linear` $\rightarrow$ `BatchNorm1d` $\rightarrow$ `ReLU` $\rightarrow$ `Dropout`

followed by a final `Linear` output layer mapping to 10 classes. The baseline configuration uses two hidden layers with widths 512 and 256, dropout probability of 0.3, and ReLU activation. The input is explicitly flattened using `nn.Flatten` as the first module.

A second implementation (`MLP2`) uses `nn.ModuleList` to store layers individually, enabling explicit per-layer iteration in the forward pass. This variant supports extracting intermediate activations for t-SNE visualization.

The total parameter count for the baseline model is approximately 535,818: $784 \times 512 + 512 = 401{,}920$ (first layer + bias), $512 \times 256 + 256 = 131{,}328$ (second layer + bias), and $256 \times 10 + 10 = 2{,}570$ (output layer + bias).

Figure 1 shows the computational graph of the baseline MLP generated using `torchviz`.

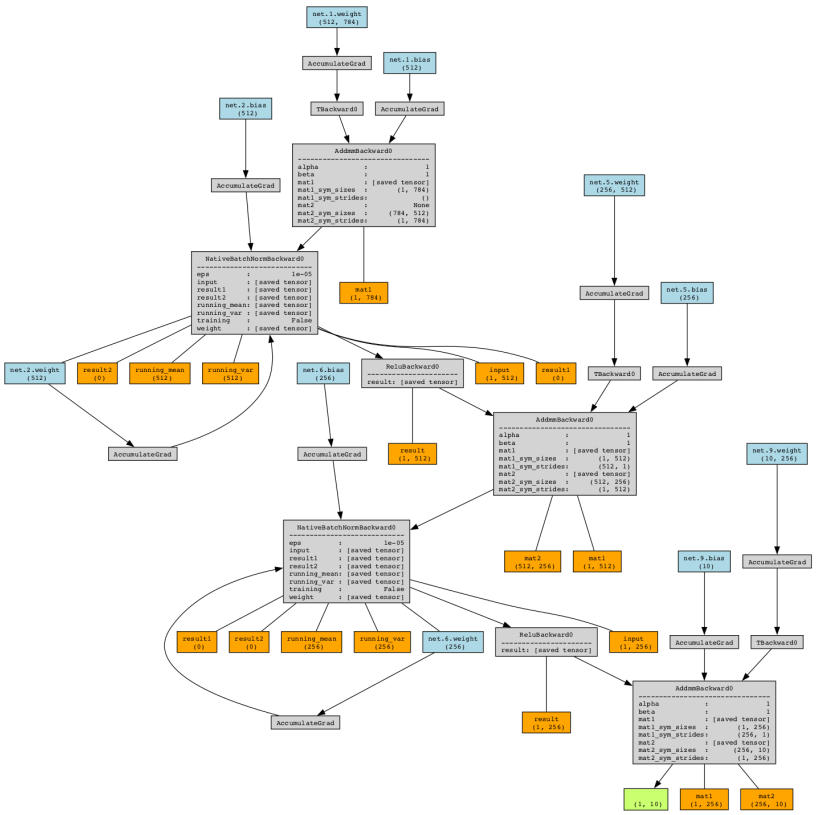

Figure 1: Computational graph of the baseline MLP ($784 \to 512 \to 256 \to 10$), visualized with `torchviz`. Orange nodes represent learnable parameters, grey nodes represent intermediate operations.

4 Training Framework

All experiments share the following training setup unless otherwise specified:

- **Optimizer:** Adam with learning rate 10^{-3} and weight decay 10^{-4}.
- **Loss function:** Cross-Entropy Loss.
- **LR Scheduler:** StepLR (step size 5, $\gamma = 0.5$).
- **Epochs:** 15.
- **Batch size:** 64.
- **Seed:** 42 (fixed across all runs for reproducibility).

The training loop tracks validation accuracy each epoch and saves the model checkpoint corresponding to the best validation accuracy. The framework supports configurable L1 regularization (added manually to the loss), multiple LR scheduler types (StepLR, CosineAnnealingLR, ReduceLROnPlateau), and early stopping with configurable patience.

5 Numerical Results and Ablations

5.1 Impact of MLP Architecture (Depth and Width)

The architecture's depth (number of hidden layers) and width (neurons per layer) significantly impact classification performance.

Table 1: Impact of network architecture on test accuracy.

Configuration	Hidden Layers	Test Accuracy
Baseline	512, 256	98.60%
Shallow	512	98.44%
Deep	512, 256, 128, 64	98.71%
Narrow	64, 32	97.43%
Wide	1024, 512	98.66%

Increasing depth to four layers yielded the highest overall accuracy of 98.71%, indicating that additional representational capacity benefits this task when sufficient width is maintained. The narrow configuration (64, 32) severely bottlenecked the model, dropping accuracy to 97.43% and exhibiting noticeably higher training loss throughout. The wide configuration (1024, 512) provided only a marginal improvement over the baseline, suggesting that 512–256 already captures sufficient complexity for MNIST. As shown in the architecture accuracy plots (Figure 3), deeper and wider models converge faster in early epochs.

5.2 Impact of Activation Functions

Table 2: Impact of activation function on test accuracy.

Activation	Test Accuracy
ReLU (Baseline)	98.60%
GELU	98.61%

Both activation functions achieved nearly identical performance. GELU's smooth approximation of ReLU provides a negligible edge on MNIST. The training and validation curves (Figure 3, Activation row) are virtually overlapping throughout training. On more complex datasets, the smoothness of GELU may provide a greater advantage by allowing more nuanced gradient flow.

5.3 Impact of the Dropout Module

Table 3: Impact of dropout probability on test accuracy.

Dropout	Test Accuracy	Final Train Loss
0.0 (No dropout)	98.61%	0.0057
0.3 (Baseline)	98.60%	0.0222
0.7	98.10%	0.1258

Removing dropout entirely (0.0) resulted in the lowest training loss (0.0057 by epoch 15), indicating heavy overfitting to the training data. Despite this, the test accuracy was comparable to the baseline, suggesting that MNIST's simplicity limits the generalization gap. A dropout of 0.3 strikes the best balance: the model trains efficiently while maintaining strong generalization. A dropout of 0.7 significantly impeded learning, as evidenced by high training loss even after 15 epochs, resulting in underfitting and a test accuracy drop to 98.10%.

5.4 Impact of Batch Normalization

Table 4: Impact of batch normalization placement on test accuracy.

Configuration	**Test Accuracy**	**Best Val Acc**
BN before activation (Baseline)	98.60%	98.60%
BN after activation	98.46%	98.46%
No BN	98.58%	98.58%

Placing batch normalization *before* the activation function yields the best results. The theoretical justification is that BN normalizes the pre-activation distribution, ensuring that inputs to the non-linear activation function are centered around zero with unit variance. This prevents the inputs from drifting into the saturating regions of the activation function (particularly relevant for sigmoid/tanh, and for the zero-gradient region of ReLU for negative inputs), which stabilizes gradients and accelerates convergence.

The empirical results confirm this: the BN-before-activation baseline achieved 96.87% validation accuracy after just 1 epoch, compared to 96.15% for BN-after-activation and 96.53% for no BN. This early convergence advantage, visible in the BN plots (Figure 4), demonstrates that pre-activation normalization provides better gradient conditioning from the start of training. The no-BN variant eventually converged to a competitive 98.58% but required more epochs to reach comparable performance.

5.5 Impact of Regularization

5.5.1 L2 Regularization (Weight Decay)

Table 5: Impact of L2 regularization on test accuracy.

Weight Decay (λ_{L2})	**Test Accuracy**
0.0	98.55%
10^{-4} (Baseline)	98.60%
10^{-2}	97.85%

L2 regularization (weight decay) penalizes large weight magnitudes by adding $\lambda_{L2} \sum \|w\|_2^2$ to the loss. The baseline penalty of 10^{-4} provides a slight improvement over no regularization. An aggressive penalty of 10^{-2} severely constrained the model's capacity, preventing it from learning complex decision boundaries and reducing accuracy to 97.85%.

5.5.2 L1 Regularization

Table 6: Impact of L1 regularization on test accuracy.

L1 Coefficient (λ_{L1})	**Test Accuracy**
0.0 (Baseline)	98.60%
10^{-5}	98.70%
10^{-4}	98.50%
10^{-3}	97.31%

L1 regularization adds $\lambda_{L1} \sum |w|$ to the loss, promoting weight sparsity. A small coefficient of 10^{-5} slightly improved performance over the baseline, likely by encouraging the network to rely on the most informative features. At 10^{-4}, the training loss remained significantly elevated throughout training (Figure 4, L1 row)

because the L1 penalty dominates the total loss. At 10^{-3}, the penalty was so severe that the model's training loss never dropped below 0.63, and test accuracy fell to 97.31%. This demonstrates that L1 regularization requires careful tuning—the useful range is much narrower than L2 for this task.

5.6 Impact of the Training Framework

5.6.1 LR Scheduler Comparison

Table 7: Impact of LR scheduler on test accuracy.

Scheduler	**Test Accuracy**	**Best Val Acc**
StepLR (Baseline)	98.60%	98.60%
CosineAnnealingLR	98.65%	98.65%
ReduceLROnPlateau	98.40%	98.40%

CosineAnnealingLR achieved the best performance among schedulers. Its smooth, gradual decay of the learning rate allows the model to make fine-grained updates in later epochs, which is visible in the scheduler validation curves (Figure 3, LR Scheduler row) where cosine annealing achieves the highest final validation accuracy. StepLR's discrete halving at epoch 5 and 10 also works well but introduces abrupt transitions. ReduceLROnPlateau underperformed because it is reactive—it only reduces the learning rate after detecting stagnation, which means it maintains a higher learning rate longer than optimal during the later stages of training.

5.6.2 Early Stopping Analysis

Examining the baseline training logs, the validation loss reaches its minimum at epoch 13 (0.0434) before showing signs of slight oscillation in epochs 14–15. The validation accuracy peaks at epoch 14 (98.60%) and does not improve further. This plateau indicates the onset of overfitting, where continued training improves training accuracy (reaching 99.27% by epoch 15) while validation performance stagnates. Implementing an early stopping mechanism with a patience of 3–5 epochs would halt training around epoch 14, saving computational resources while preserving the best generalization performance.

5.7 t-SNE Feature Visualization

To understand how the trained network separates digit classes in its learned feature space, t-SNE was applied to the penultimate layer activations of the baseline model on the test set. Figure 2 shows the resulting 2D projection.

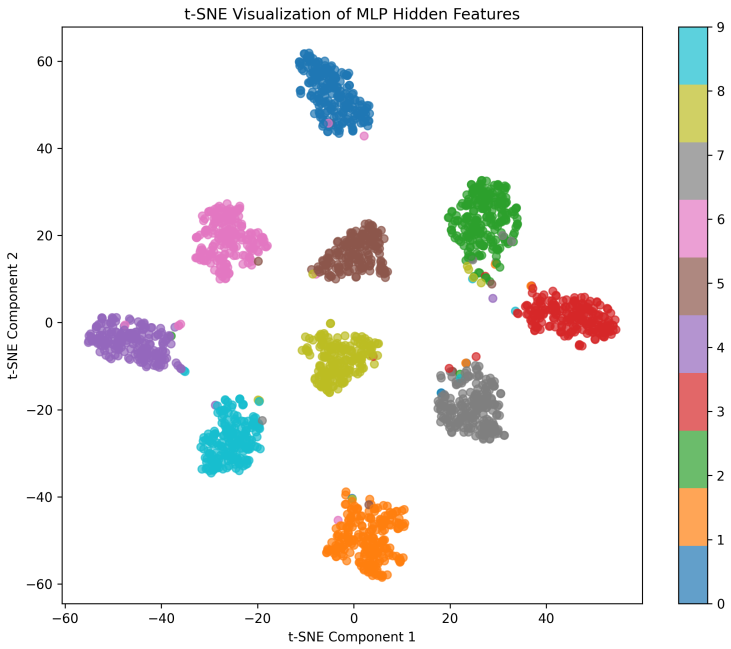

Figure 2: t-SNE visualization of MLP hidden features (penultimate layer) on MNIST test set. Each color represents a digit class (0–9). The well-separated clusters confirm that the MLP learns discriminative representations.

The visualization reveals 10 distinct, well-separated clusters corresponding to each digit class. Notably, digits with similar visual structures (such as 4 and 9, or 3 and 5) have clusters that are positioned closer together in the embedding space, reflecting the learned feature similarity. The tight clustering with minimal overlap is consistent with the high test accuracy of 98.60%.

6 Training and Validation Curves

Figures 3 and 4 present the complete set of training vs. validation accuracy and loss curves across all seven ablation groups.

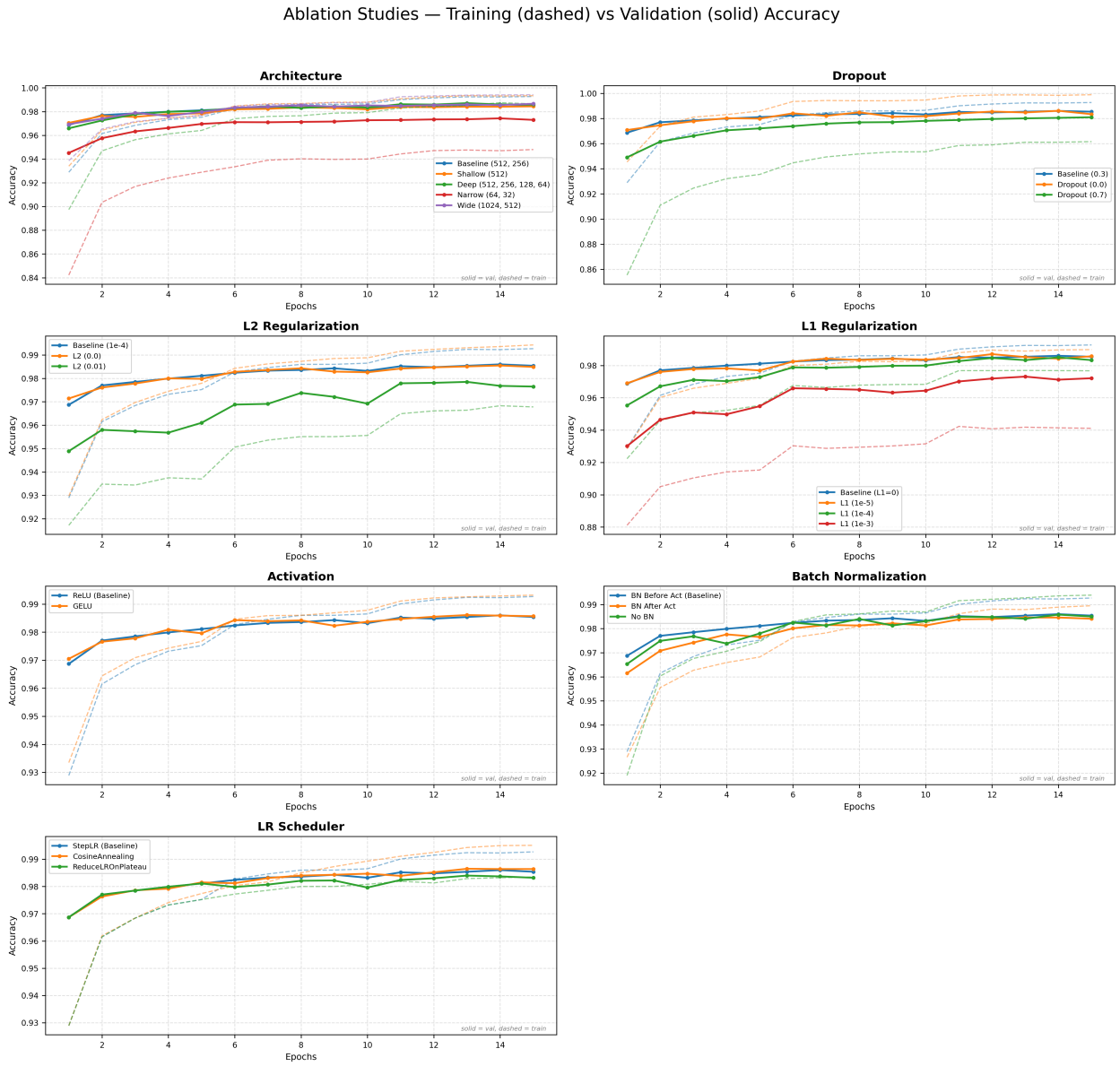

Figure 3: Training and validation accuracy curves for all ablation studies across 15 epochs. Each row corresponds to one ablation group: Architecture, Dropout, L2, L1, Activation, Batch Normalization, and LR Scheduler.

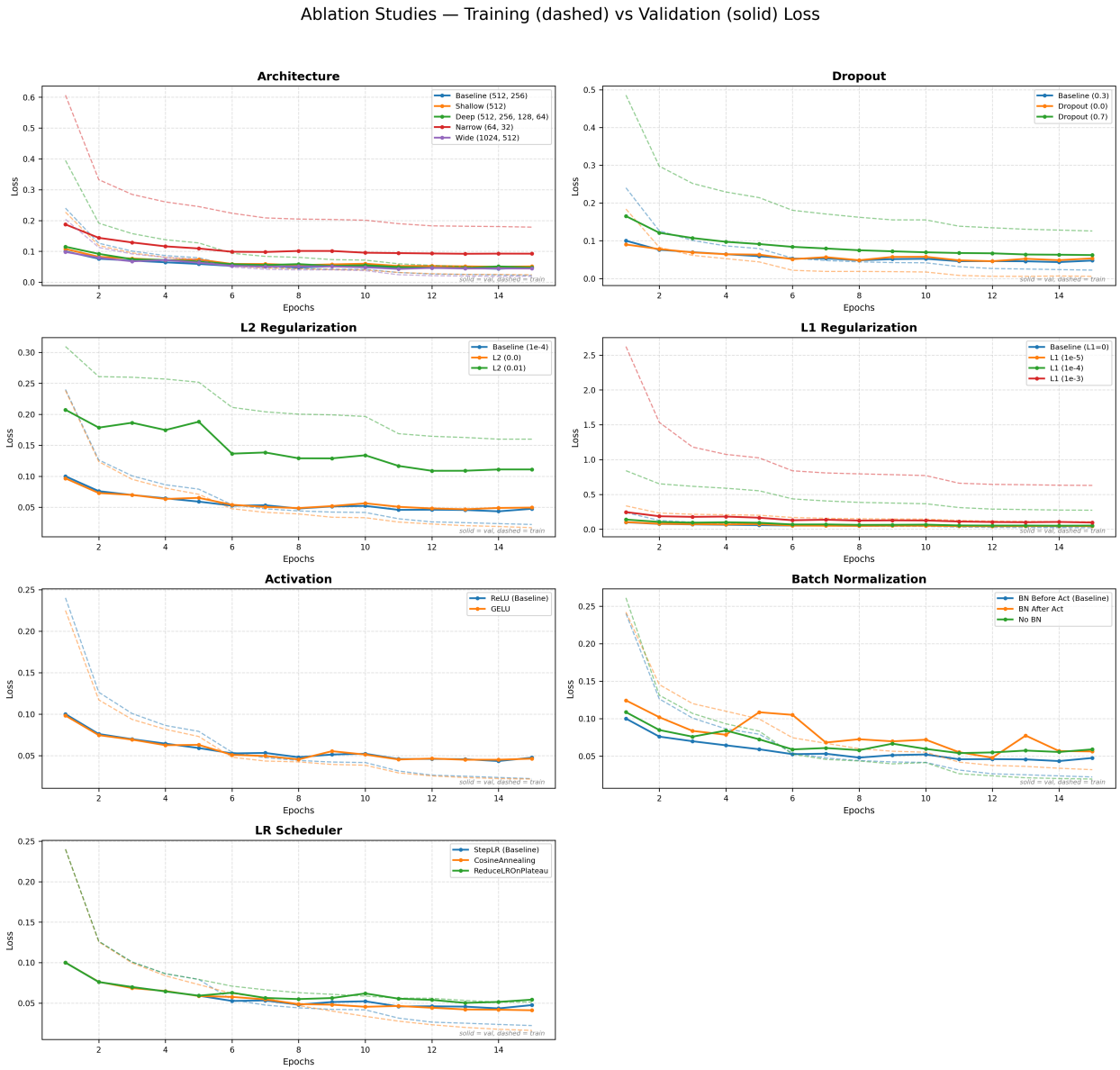

Figure 4: Training and validation loss curves for all ablation studies across 15 epochs. The L1 row notably shows how aggressive L1 penalties inflate total training loss while validation loss remains comparatively lower.

7 Summary

Table 8: Summary of all ablation results. Bold indicates the best configuration per group.

Ablation Group	Best Configuration	Test Accuracy
Architecture	**Deep (512, 256, 128, 64)**	**98.71%**
Activation	GELU (marginal)	98.61%
Dropout	0.0 / 0.3 (comparable)	98.61% / 98.60%
Batch Normalization	**BN before activation**	98.60%
L2 Regularization	10^{-4}	98.60%
L1 Regularization	10^{-5}	**98.70%**
LR Scheduler	**CosineAnnealingLR**	**98.65%**

The experiments demonstrate that MNIST classification with an MLP is largely robust to hyperparameter choices—most reasonable configurations achieve >98% accuracy. The most impactful factors are architectural capacity (narrow networks suffer) and regularization strength (excessive L1, L2, or dropout severely impair learning). Batch normalization placed before activation provides the best gradient conditioning, and CosineAnnealingLR offers a slight edge over step-based scheduling. The deep 4-layer architecture with light L1 regularization (10^{-5}) represents the strongest single configuration observed.